A Microservices-Based Home Energy Tracker for Real-Time Monitoring, Intelligent Alerting, and AI-Driven Energy Insights

Rohit Shrivastava[1] , Anshika Rai[2] , Harsh Gupta[3] , Syamasudha Veeragandham[4]
[1,2,3,4]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India – 632014

Email: [1]rohit.lalbahadur2023@vitstudent.ac.in, [2]anshika.rai2023@vitstudent.ac.in, [3]harsh.gupta2023@vitstudent.ac.in, [4] syamasudha.v@vit.ac.in

[4]Corresponding author

Abstract

Residential energy-monitoring platforms require a traceable path from device telemetry to actionable feedback, but existing solutions frequently address sensing, analytics, or scalable software deployment separately. This work presents a microservices-based Home Energy Tracker that integrates simulated device telemetry, event-driven processing, time-series storage, deterministic energy analytics, and threshold-based alerts. The implemented prototype consists of Spring Boot User, Device, Ingestion, Usage, Insight, and Alert services behind an API Gateway. A Wokwi ESP32 virtual meter produces timestamped JSON telemetry, which the Ingestion Service validates, publishes to Apache Kafka, and the Usage Service stores in InfluxDB. MySQL maintains household and device metadata, while the Insight Service calculates a slab-based bill forecast, appliance ranking, and a standby-load heuristic. An optional Ollama component generates narrative wording only; numerical outputs remain deterministic and fall back safely when the AI component is unavailable. A controlled local functional run accepted three synthetic readings totalling 1.50 kWh, returned the expected forecast and standby values, and produced a threshold alert captured by Mailpit. These results demonstrate functional integration, not physical-meter accuracy, predictive-model accuracy, scalability, or resilience. The architecture provides a reproducible foundation for future calibrated sensing and empirical evaluation.

Keywords:

Home Electrical Energy Monitoring; Internet of Things; microservices; Spring Boot; Apache Kafka; InfluxDB; deterministic energy analytics; Wokwi simulation; threshold alerting.

1. INTRODUCTION

Connected appliances and digital services increasingly shape residential electricity use. However, monthly bills alone provide limited visibility into when energy is consumed, which device contributes most, or whether a configured household threshold has been exceeded. A practical Home Energy Management System therefore needs an auditable route from telemetry to stored records, analytical output, and user notification.

Existing research provides strong but often separate contributions in low-cost sensing, anomaly detection, time-series analysis, and distributed architectures. A software prototype that links these concerns must avoid treating simulated telemetry as calibrated measurement data or treating rule-based outputs as a trained artificial-intelligence model. This distinction is central to the experimental claims of this work.

The proposed Home Energy Tracker implements independent Spring Boot services for household profiles, devices, telemetry ingestion, time-series usage, insights, and alerts. A Wokwi ESP32 simulator emits deviceId, energyConsumed, and timestamp records. The API Gateway routes the record to the Ingestion Service, Kafka decouples ingestion from downstream work, InfluxDB stores the time-series point, and the Insight Service returns deterministic forecast, ranking, and standby outputs. The Alert Service persists a threshold-breach alert and sends an email to a local Mailpit inbox.

The study is scoped as a system-integration and functional-verification project. It reports a controlled local execution, source-backed formulas, and automated test outcomes. Physical meter calibration, authentication/authorization, anomaly classification, benchmark comparisons, resilience testing, and large-scale performance measurements remain future work rather than completed claims.

2. OBJECTIVES

The objectives of the proposed Home Energy Tracker are:

1. Design a modular, event-driven microservices architecture for household energy monitoring using Spring Boot and an API Gateway.
2. Accept timestamped device telemetry through a validated REST ingestion interface and transfer accepted events through Apache Kafka to a time-series store.
3. Provide transparent, deterministic energy-management outputs: a slab-based bill forecast, appliance ranking, and standby-load heuristic.
4. Implement a configurable threshold-alert path that persists alert history and demonstrates local email delivery through Mailpit.
5. Validate the implemented path through automated tests and controlled end-to-end runs, while documenting the limits of simulation-based evidence.
6. Maintain an extensible foundation for future calibrated sensing, secure access control, data-driven anomaly detection, and resilience evaluation.

2.1 LITERATURE REVIEW

1 Smart Metering Meets AI: Real-Time Appliance Monitoring and Anomaly Detection Over DDS Middleware (2025)

This study proposes an intelligent smart metering framework that integrates real-time energy sensing, decentralized communication, and artificial intelligence for appliance monitoring and anomaly detection. The proposed architecture utilizes eProsima Fast Data Distribution Service (Fast DDS), a broker-less publish-subscribe middleware, to transmit high-resolution electrical measurements from smart meters to a remote server with minimal communication latency. A multi-task deep neural network is employed to simultaneously classify household appliances and detect abnormal operating conditions using short time-series windows of electrical power data.

Experimental evaluation demonstrated over 93% accuracy in appliance classification and more than 99% accuracy in anomaly detection while maintaining an average communication latency of approximately

25 milliseconds. These results indicate that the proposed architecture is suitable for real-time smart energy monitoring applications requiring high throughput and reliable communication.

Although the proposed system provides excellent real-time monitoring and AI-driven anomaly detection, its architecture mainly focuses on appliance classification and distributed communication. The study provides limited discussion on scalable backend service management, user authentication, intelligent alert generation, and long-term historical energy analytics within a complete home energy management platform.

This study motivates the use of event-driven telemetry and rich analysis. In the present prototype, Apache Kafka and InfluxDB support the integration path, while trained appliance classification, anomaly detection, and secure authentication are not implemented and remain future work.

2 Real-Time IoT-Based Energy Power Consumption Monitoring System for Smart Homes (2025)

This research presents a low-cost Internet of Things (IoT)-based energy monitoring system designed for residential smart homes. The proposed prototype employs a PZEM-004T energy sensor and an ESP8266 microcontroller to measure electrical parameters including voltage, current, power, energy consumption, frequency, and power factor. The collected information is displayed locally using an OLED display and remotely through a Blynk-based mobile application.

Experimental results indicate a measurement error of less than 0.05%, demonstrating high monitoring accuracy and improved power factor through automated capacitor bank control. The study highlights the effectiveness of combining affordable sensing hardware with remote monitoring capabilities to improve household energy awareness and reduce electricity consumption.

Despite providing accurate real-time monitoring, the proposed solution primarily focuses on hardware implementation and remote visualization. It does not include intelligent backend services, AI-driven energy analytics, scalable microservices architecture, secure authentication mechanisms, or advanced event-driven communication suitable for large-scale deployments.

This hardware-oriented study motivates future physical sensing. The current prototype instead uses Wokwi-generated test input to exercise the backend interface; it does not claim PZEM-level measurement accuracy or calibrated electrical parameters.

3 Design of a Microservices-Based Architecture for Residential Energy Efficiency Monitoring (2024)

This study proposes a distributed microservices architecture for residential energy efficiency monitoring to overcome the limitations of traditional service-oriented architectures. The system is designed to improve scalability, fault tolerance, maintainability, and interoperability while enabling efficient monitoring and control of residential electrical devices. Cloud-based services are utilized to process energy information and improve overall system performance.

The proposed architecture demonstrates significant improvements in energy efficiency management, demand response, power quality, and system scalability. The modular design allows independent deployment and maintenance of individual services, making the system more flexible than conventional monolithic architectures. The authors also recommend integrating advanced machine learning techniques in future work to improve intelligent energy management further.

Although the architecture successfully addresses scalability and modularity, it does not provide detailed implementation of AI-powered energy recommendations, secure authentication, intelligent alert generation, or event-driven streaming using modern messaging platforms.

This work provides an architectural basis for service separation. The present implementation adopts Spring Boot microservices, Kafka, InfluxDB, deterministic insight functions, and a threshold alert path; secure authentication and trained AI analytics are outside the current scope.

4 A Cloud Architecture for Home Energy Management Systems: A Conceptual Model (2025)

This paper presents a cloud-based conceptual architecture for Home Energy Management Systems (HEMS) that focuses on the efficient organization, ingestion, processing, storage, and analysis of IoT-generated energy data. The proposed framework divides the system into three interconnected environments: Ingestion, Operational, and Analytical. Each environment is responsible for specific tasks such as collecting real-time data, supporting operational services, and enabling large-scale analytics and decision-making. The architecture also aligns system functionalities with the needs of different stakeholders, including homeowners, service providers, and utility operators.

The proposed architecture demonstrates that separating data ingestion, operational processing, and analytical functions improves system scalability, adaptability, and maintainability. By organizing software components around stakeholder requirements, the framework provides a structured foundation for real-time monitoring, data management, and decision support in modern Home Energy Management Systems.

However, the study primarily presents a conceptual software architecture rather than a complete implementation. It does not describe an event-driven communication mechanism such as Apache Kafka, AI-powered energy insight generation, secure authentication using Keycloak and JWT, or intelligent real-time alert services for abnormal energy consumption.

The layered conceptual model informs the separation of ingestion, operational services, and analytics in the proposed prototype. The implemented additions are API routing, Kafka event transfer, InfluxDB storage, deterministic insights, and local alert delivery; Keycloak authentication is reserved for future development.

5 Smart Metering Enabled by the Internet of Things for Monitoring Residential Energy Consumption and Power Quality (2025)

This paper presents an Internet of Things (IoT)-enabled smart metering system designed for continuous monitoring of residential energy consumption and power quality. The proposed solution utilizes an ESP32 microcontroller along with ZMPT101B voltage sensors and SCT013 current sensors to capture high-resolution electrical measurements. The collected data is analyzed to detect and classify various power quality disturbances, including voltage sags, voltage swells, harmonic distortions, spikes, notches, and transient events. The study emphasizes the importance of combining real-time sensing with intelligent monitoring to improve energy efficiency and maintain electrical reliability in residential environments.

The experimental results demonstrate that the proposed system can accurately monitor electrical parameters while effectively identifying different categories of power quality disturbances. The authors also suggest that integrating machine learning algorithms with the monitoring platform can further enhance prediction accuracy, optimize energy consumption patterns, and support smarter energy management in future smart home environments.

Although the proposed system provides comprehensive monitoring of electrical parameters and power quality events, it mainly concentrates on sensing hardware and disturbance classification. The study does not describe a scalable microservices backend, event-driven communication using messaging platforms, secure user authentication, historical time-series analytics, or AI-powered personalized energy recommendations.

The study underlines the importance of accurate metering and power-quality sensing. It therefore motivates a future calibrated hardware phase, while the current Wokwi simulation is used only for software integration and functional testing.

6 Anomaly Detection in Smart Electric Meters for Detecting Faults and Misuse of Electric Energy Consumption (2025)

This study proposes a hybrid anomaly detection framework for identifying faults, energy misuse, and abnormal electricity consumption patterns in smart electric meters. The authors address the challenge of detecting anomalies within the large volume of data generated by smart meters by combining multiple statistical and machine learning techniques. The proposed framework integrates Z-score analysis, Local Outlier Factor (LOF), One-Class Support Vector Machine (SVM), and Isolation Forest (iForest). An event is classified as anomalous only when all four methods agree, thereby reducing false positives and improving detection reliability.

To enhance user interaction, the framework is implemented through an interactive Streamlit dashboard that provides real-time visualization of energy consumption, peak-time alerts, usage forecasting, and personalized energy-saving recommendations. Experimental results indicate that the hybrid approach performs better than individual anomaly detection techniques, offering improved detection accuracy and practical applicability for smart grid and microgrid environments. The study demonstrates that combining complementary detection algorithms with proactive user feedback can significantly improve energy management and reduce unnecessary electricity consumption.

Despite its strong anomaly detection capability, the proposed framework mainly focuses on analytical models and dashboard visualization. It does not describe a distributed microservices architecture, event-driven communication, secure authentication, or dedicated backend services for scalable deployment across multiple users and households.

The anomaly-detection literature motivates future analytical extensions. In the current implementation, the Alert Service applies a configured threshold rule to aggregated usage rather than a hybrid anomaly classifier.

7 A Scalable IoT–AI Framework for Real-Time Energy Anomaly Detection in Smart Grid Networks (2026)

This paper proposes a scalable IoT–AI framework for detecting energy anomalies in smart grid environments. The architecture consists of four layers: IoT-based data acquisition, intelligent preprocessing, machine learning analytics, and operator decision support. Multiple machine learning models are integrated to identify abnormal consumption behaviour, while Explainable Artificial Intelligence (XAI) using SHAP provides interpretable explanations for each prediction.

The proposed framework is evaluated using the SGCC benchmark dataset containing more than 42,000 consumers. Experimental results demonstrate 94.3% accuracy, 92.8% precision, 93.5% recall, and 93.1%

F1-score while maintaining an average alert latency below 2.5 seconds. These results indicate that AI-driven anomaly detection can significantly improve the reliability of large-scale smart energy systems.

Although the framework achieves excellent anomaly detection performance, it mainly focuses on AI analytics for smart grids. It does not describe a complete backend architecture with microservices, API Gateway, authentication, or time-series data management.

This framework demonstrates the potential of AI-assisted anomaly detection at scale. The present work addresses service integration at prototype scale; it does not implement or evaluate a trained anomaly model, accuracy, precision, recall, or F1-score.

8 Detection of Anomalies in Daily Activities Using Data from Smart Meters (2024)

This study investigates the use of smart meter data to detect anomalies in the daily activities of residential users. By analysing electricity consumption patterns over time, the proposed approach identifies behavioural deviations that may indicate appliance faults, abnormal energy usage, or unexpected occupant activities. The research highlights the importance of continuous monitoring and data-driven anomaly detection for improving residential energy efficiency.

The experimental evaluation demonstrates that analysing daily consumption behaviour can successfully distinguish normal and abnormal usage patterns while reducing unnecessary energy wastage. The study also emphasizes that historical energy data plays a significant role in improving anomaly detection accuracy.

However, the research mainly focuses on behavioural analysis and does not provide a scalable backend implementation, secure authentication, or AI-generated recommendations for end users.

The study supports the value of historical time-series data. The current prototype stores and queries historical usage but does not infer unusual daily behavior; its active alert condition is a configurable threshold.

2.9 Enhancing Resilience in Complex Energy Systems Through Real-Time Anomaly Detection: A Systematic Literature Review (2024)

This systematic literature review investigates the role of anomaly detection in improving the resilience of modern energy systems. The study analyses various anomaly detection techniques, resilience strategies, and operational challenges across complex energy infrastructures.

The review identifies several research gaps, including limited resilience validation, insufficient benchmarking, and the absence of standardized evaluation methodologies for real-time energy management systems.

Although the paper provides valuable research directions, it does not propose an implementation framework or complete software architecture.

This review motivates future resilience experiments. The current service separation and asynchronous messaging supply a design basis, but no fault injection, chaos testing, or resilience result is claimed in this paper.

2.10 Anomaly Detection and Root Cause Analysis of Microservices Energy Consumption (2024)

This paper focuses on detecting anomalies in the energy consumption of microservices applications and identifying their root causes. The proposed approach analyses resource utilization patterns to discover inefficient services and abnormal operational behaviour.

The experimental evaluation demonstrates that anomaly detection can improve resource utilization and simplify performance optimization in distributed software systems.

However, the study concentrates on software microservices rather than residential energy monitoring and does not include IoT integration or consumer-focused analytics.

This work informs the importance of monitoring distributed services. The present prototype applies microservice separation to residential energy telemetry and deterministic user-facing outputs; advanced anomaly analysis remains future work.

2.2 RESEARCH GAP

Although recent studies have made significant progress in IoT-based energy monitoring, anomaly detection, Artificial Intelligence, and microservices architectures, several important research gaps remain. These limitations reduce the effectiveness of current Home Energy Management Systems in terms of scalability, reliability, intelligence, and real-world deployment. Based on the reviewed literature, the following research gaps have been identified.

Gap 1 – Lack of Standardised Benchmark Datasets and Evaluation Metrics

One of the major challenges identified in the literature is the absence of standardized benchmark datasets and unified evaluation metrics for home energy anomaly detection. Most existing studies evaluate their algorithms using different datasets, experimental environments, and performance measures, making it difficult to compare results fairly. Furthermore, several studies highlight the shortage of annotated datasets and clearly defined ground truth labels for abnormal power consumption.

Supporting Papers:

• Artificial Intelligence-Based Anomaly Detection of Energy Consumption in Buildings (2021)

• IoT Anomaly Detection Methods and Applications: A Survey (2022)

• Anomaly Detection and Root Cause Analysis of Microservices Energy Consumption (2024)

Research Opportunity:

Future systems should support standardized evaluation methodologies and utilize consistent datasets for reliable comparison of anomaly detection techniques.

Evidence: The uploaded research-gap document explicitly identifies the absence of standardized benchmarks, annotated datasets, unified metrics, and clear anomaly definitions as a major unresolved issue.

Gap 2 – Limited Context-Aware Energy Analysis

Most existing anomaly detection systems identify abnormal energy consumption by analyzing electrical parameters alone. However, they often ignore contextual information such as occupancy behaviour, environmental conditions, appliance usage schedules, or user activities. The absence of contextual awareness increases false-positive alerts because unusual electricity usage may still represent normal user behaviour under certain circumstances.

Supporting Papers:

• Artificial Intelligence-Based Anomaly Detection of Energy Consumption in Buildings (2021)

• Detection of Anomalies in Daily Activities Using Data from Smart Meters (2024)

• IoT Anomaly Detection Methods and Applications: A Survey (2022)

Research Opportunity:

Future Home Energy Management Systems should incorporate richer contextual information to improve anomaly detection accuracy and reduce unnecessary alerts.

Evidence: The uploaded gap analysis identifies missing contextual awareness as a key cause of false positives in current studies

Gap 3 – Insufficient Validation of Microservice Resilience

Although many recent studies recommend microservices because of their scalability and fault tolerance, very few validate these claims through resilience testing. Most systems are evaluated under controlled laboratory environments rather than realistic production scenarios involving service failures, communication interruptions, or cascading faults. Consequently, the practical resilience of many proposed architectures remains uncertain.

Supporting Papers:

• Edge Computing and Microservices Middleware for Home Energy Management Systems (2022)

• Enhancing Resilience in Complex Energy Systems Through Real-Time Anomaly Detection (2024)

• A Microservices-Based Solution with Hybrid Communication for Energy Management in Smart Grid Environments (2026)

Research Opportunity:

Future research should investigate resilience validation techniques, fault injection experiments, and large-scale deployment testing to verify the robustness of microservices-based energy platforms.

Evidence: The uploaded research-gap document calls this the most striking gap, noting the near absence of resilience testing and chaos engineering despite frequent claims of fault tolerance.

Gap 4 – Limited Privacy-Preserving AI Integration

Many existing energy monitoring systems successfully perform anomaly detection and predictive analytics but provide limited discussion on privacy-preserving AI architectures. Most solutions depend on centralized processing of user data without considering privacy concerns associated with residential energy consumption. The integration of secure authentication, local AI inference, and privacy-aware system design remains relatively limited.

Supporting Papers:

• Artificial Intelligence Based Anomaly Detection of Energy Consumption in Buildings (2021)

• IoT Anomaly Detection Methods and Applications: A Survey (2022)

• Smart Metering Meets AI: Real-Time Appliance Monitoring and Anomaly Detection Over DDS Middleware (2025)

Research Opportunity:

Future systems should investigate secure authentication mechanisms and privacy-conscious AI deployment for intelligent residential energy management.

Evidence: The uploaded research-gap summary lists insufficient integration of privacy-preserving, real-time architectures as one of the unresolved limitations.

Gap 5 – Limited Integration of Scalable AI with Event-Driven Microservices

Current literature generally addresses either scalable microservices architecture or AI-based anomaly detection independently. Very few studies combine distributed microservices, real-time event streaming, time-series databases, intelligent alert generation, and AI-powered decision support into a unified backend platform for residential energy monitoring.

Supporting Papers:

• Smart Metering Meets AI (2026)

• Design of a Microservices-Based Architecture for Residential Energy Monitoring (2024)

• A Cloud Architecture for Home Energy Management Systems (2025)

• A Microservices-Based Solution with Hybrid Communication for Energy Management in Smart Grid Environments (2026)

Research Opportunity:

Developing a unified platform that integrates event-driven microservices, secure authentication, historical time-series analytics, AI-generated insights, and intelligent alerting remains an open research direction for Home Energy Management Systems.

Evidence: This gap is consistent with the uploaded evidence showing support for real-time monitoring, anomaly detection, and scalable microservices individually, while motivating an integrated Home Energy Tracker architecture.

The present prototype directly addresses Gap 5 at the system-integration level by linking simulated telemetry, event-driven microservices, time-series storage, deterministic decision support, and alert delivery. Gaps concerning standardized datasets, contextual analysis, resilience validation, and privacy-preserving access control are not resolved; they are retained as explicit limitations and future work.

2.3 NOVELTY OF THE PROPOSED WORK

The novelty of the work is the implementation and traceable verification of a compact residential-energy prototype that combines a Wokwi telemetry interface, independently deployable Spring Boot services, Kafka event transfer, InfluxDB time-series storage, deterministic insight computation, and a threshold-alert path in one workflow. The contribution is not a new anomaly-detection algorithm, a calibrated smart meter, or a validated AI predictor. Instead, it provides an auditable integration baseline in which each numerical dashboard value can be linked to a defined formula and each alert can be linked to an explicit threshold condition.

2.4 NOVEL CONTRIBUTIONS

- An event-driven microservices architecture with User, Device, Ingestion, Usage, Insight, Alert, and API Gateway responsibilities.

- A Wokwi ESP32 simulation interface that sends the same timestamped JSON payload shape used by the ingestion API.
- Kafka-based decoupling of ingestion from time-series persistence, with InfluxDB used for device-tagged energy_usage records.
- Transparent deterministic calculations for bill forecasting, appliance ranking, and standby-load estimation.
- A scheduled threshold-alert flow that persists alert records and demonstrates local email delivery through Mailpit.
- A reproducible controlled verification protocol and evidence record for a 1.50 kWh analytics case and a 1.00 kWh threshold-breach case.

3. PROPOSED METHODOLOGY AND PROPOSED SYSTEM

3.1 Overall Methodology

The proposed work is an event-driven software-system prototype for household energy monitoring. It is designed primarily to address the integration gap identified in Section 2: the need to combine device telemetry, scalable service separation, time-series storage, deterministic energy analytics, and threshold-based notification within a single residential energy-management platform. The present implementation is not a supervised machine-learning experiment. Its current energy input is a controlled Wokwi ESP32 simulation, and its numerical outputs are generated by deterministic calculations.

The input to the system is an energy-telemetry record containing deviceId, energyConsumed, and timestamp. In the current prototype, the Wokwi firmware produces this JSON record every 8 s. The virtual potentiometer and three modes generate controlled scenarios: Eco (approximately 1-6 kWh), Heavy (approximately 8-20 kWh), and Alert Test (approximately 26-40 kWh). These values are scenario inputs for integration testing and must not be interpreted as calibrated physical meter measurements.

Data acquisition begins when the virtual ESP32 sends a REST request through the API Gateway to the Ingestion Service. The service validates that deviceId and timestamp are present and that energyConsumed is non-negative. It then transforms the request into an energy-usage event and publishes that event to Apache Kafka. The Usage Service consumes the event, stores deviceId as a tag and energyConsumed as a field in the InfluxDB energy_usage measurement, and preserves the event timestamp for time-window queries.

Feature transformation is performed through aggregation rather than feature learning. For a selected user and observation window of D days, the Usage Service sums energyConsumed for each registered device. The Insight Service derives household total energy, daily consumption rate, projected 30-day energy, slab-based cost, device contribution percentage, estimated device cost, and a standby-load estimate. The Alert path separately aggregates the preceding one-hour usage window every 10 s, maps device readings to their users, and compares each household total with its configured energy threshold.

The outputs are (i) device-level and household-level usage summaries, (ii) a bill forecast, (iii) ranked appliances, (iv) a standby-risk estimate with recommendations, and (v) persisted alert records with local email capture through Mailpit. Evaluation at the current review stage focuses on functional correctness and end-to-end traceability: accepted ingestion request, Kafka event propagation, InfluxDB persistence, insight retrieval, and alert delivery. One controlled functional run is reported in Section 5.1. Quantitative

performance, forecast-error, and classification metrics still require larger experiments and ground-truth data.

3.2 Proposed System Architecture

Fig. 1 presents the proposed architecture. The architecture separates input, event processing, storage, analytics, and presentation responsibilities. The separation allows the telemetry-facing ingestion path to acknowledge valid readings without waiting for aggregation, analytics, or notification delivery.

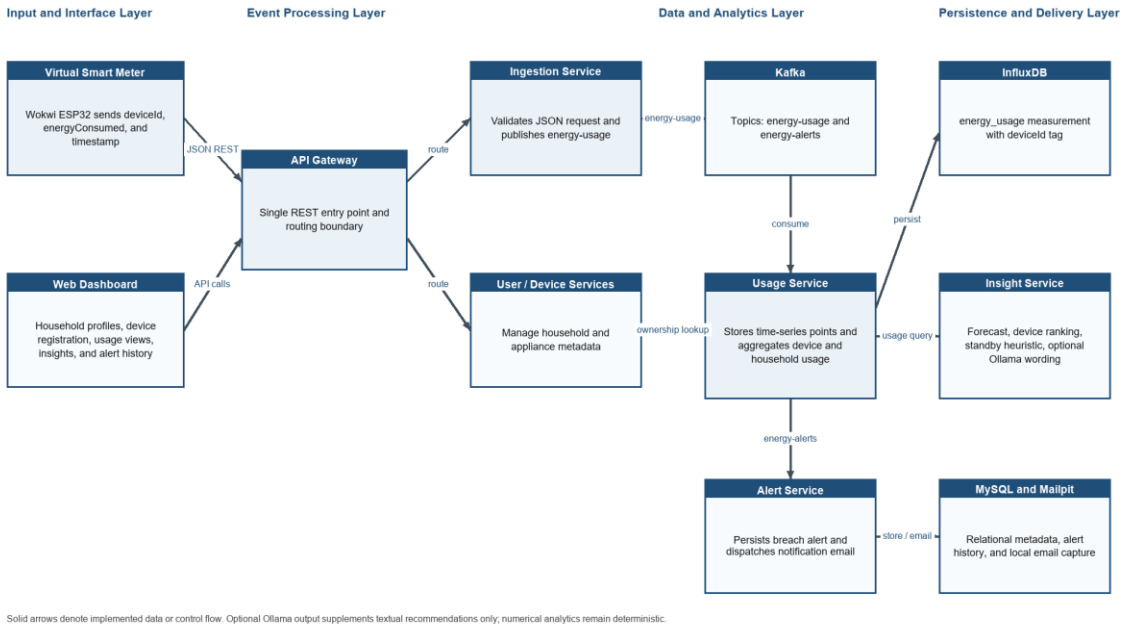


Fig. 1. Proposed system architecture of the Home Energy Tracker.

The virtual smart-meter block represents the current Wokwi ESP32 implementation. It supplies the same REST payload shape expected from a future physical meter but does not replace calibrated sensing hardware. The API Gateway is the public routing boundary for the browser and virtual meter. The Ingestion Service validates the payload and emits the energy-usage event. Kafka decouples this producer from the Usage Service, which persists time-series points to InfluxDB.

User and Device Services maintain household and appliance metadata in MySQL. During aggregation, the Usage Service retrieves device ownership and user alert settings. The Insight Service queries aggregated usage data and produces deterministic forecasts, rankings, and standby estimates. The Alert Service consumes energy-alerts events, persists alert history in MySQL, and sends a notification through the configured SMTP endpoint; Mailpit is used as a local demonstration inbox. Ollama, when reachable, is used only to express recommendations in natural language. A deterministic fallback is returned if the optional AI service is unavailable.

3.3 Detailed Proposed Approach

The proposed approach is executed as the following connected stages. Each stage produces an output consumed by the next stage, thereby making the source of dashboard values and alerts traceable.

Table 1. Detailed proposed approach and component interfaces.

Component	Purpose	Input	Processing technique	Output and next stage
Profile and device registration	Associate a household with its appliances and alert setting.	User details, device name, type, location, and threshold.	REST CRUD operations stored in MySQL.	User and device identifiers used by usage aggregation.
Wokwi telemetry acquisition	Generate controlled end-to-end test telemetry.	Potentiometer and selected Eco, Heavy, or Alert Test mode.	ESP32 firmware maps analog mode state to a kWh value and creates a timestamped JSON request every 8 s.	{deviceId, energyConsumed, timestamp} sent to API Gateway.
Ingestion	Validate and decouple device-facing requests.	Telemetry JSON payload.	Bean validation: non-null deviceId/timestamp and non-negative energyConsumed; event mapping and Kafka publication.	energy-usage event for Usage Service.
Usage storage and aggregation	Persist readings and derive user/device window sums.	Kafka event plus user/device metadata.	Write InfluxDB point; Flux time-window sum grouped by deviceId; ownership resolution through Device Service.	Per-device energy totals and household totals.
Insight computation	Convert usage totals into interpretable decision support.	Aggregated usage over selected days and configured tariff values.	Daily-rate forecast, slab-cost calculation, descending device sort, 20% average-load standby heuristic.	Forecast, appliance ranking, standby risk, recommendations.
Threshold alerting	Notify a household when preceding one-hour summed usage is over its setting.	One-hour household aggregate, threshold, and alerting flag.	Scheduled rule evaluation every 10 s and energy-alerts publication on breach.	Persisted alert and local email notification.
Evaluation	Demonstrate traceability and quantify performance when experimental data are collected.	HTTP logs, Kafka records, InfluxDB query results, alert history, timing data.	Controlled functional checks: four accepted JSON events, usage and insight queries, persisted alert evidence, and archived verification output.	Evidence table and results/discussion inputs.

3.4 Algorithms and Techniques

The implemented system uses deterministic rules and mathematical transformations rather than a trained prediction or anomaly-detection model. Therefore, accuracy, F1-score, and train-test split claims are not

appropriate for the current prototype. The following techniques are implemented or directly represented in the source code.

3.4.1 Event-driven ingestion and time-window aggregation

Purpose: accept a validated telemetry request and make it asynchronously available to downstream processing. Input: deviceId, energyConsumed, and timestamp. Working principle: the Ingestion Service maps the valid request to an energy-usage Kafka event. The Usage Service consumes the event and writes an InfluxDB point with deviceId as a tag, energyConsumed as a field, and the supplied timestamp. For retrieval, a Flux query restricts the time range, filters the energy_usage measurement and energyConsumed field, groups by deviceId, and sums the values. Output: per-device energy totals for a selected user and time range. This approach is appropriate because telemetry storage and aggregation can be performed independently of client request acknowledgement.

3.4.2 Slab-based bill forecast

Purpose: estimate a 30-day energy quantity and cost from observed usage. Let E_tot be the summed energy over D selected days. The system computes the average daily rate and 30-day projection as follows:

$$E_day = E_tot / D, \quad E_30 = 30 \times E_day$$

The projected cost is calculated using configured tariff slabs. For slab limits L1 and L2, slab rates r1, r2, r3, and fixed charge F, the implementation allocates consumption sequentially to the slab ranges and adds F only to the projected monthly cost. The local prototype defaults are F = 10.00, L1 = 100 kWh, r1 = 0.12, L2 = 300 kWh, r2 = 0.18, and r3 = 0.25. These fixed demonstration parameters were used to verify the slab-calculation logic and are not represented as a utility-approved billing tariff.

Output: current-period estimated cost, daily rate, projected 30-day kWh, and projected 30-day cost. The method is appropriate for a transparent prototype because every estimate can be reproduced from the aggregated input and tariff configuration. It is not a substitute for a utility-bill forecast validated against real billing data.

3.4.3 Appliance ranking

Purpose: identify devices that contribute the largest proportion of observed consumption. Input: device-level sums for a selected time window. The devices are sorted in descending order of energyConsumed. For device i, contribution is calculated as $p_i = (E_i / E_tot) \times 100$ when E_tot is positive. Estimated device cost is allocated proportionally as $c_i = p_i \times C_tot / 100$, where C_tot is the current-period slab-based cost. Output: a ranked device list with kWh, contribution percentage, and estimated cost. This method is appropriate because the application already records device-associated telemetry and does not claim appliance disaggregation from a single aggregate meter.

3.4.4 Standby Energy Detective heuristic

Purpose: translate continuous low-level demand into an explainable estimate of potential standby cost. The current implementation does not detect night-time or low-activity periods directly. Instead, it estimates baseline power as 20% of the average load across the selected observation period. If the selected window has D days and total energy E_tot, then:

$$P_avg = (E_tot / (24D)) \times 1000 \text{ W}, \quad P_base = 0.20 \times P_avg$$

$$E_standby_day = 24 \times P_base / 1000, \quad E_standby_month = 30 \times E_standby_day$$

The system labels standby risk as LOW when P_base is at most 50 W, MEDIUM when it is greater than 50 W and at most 150 W, and HIGH when it is greater than 150 W. It calculates a monthly standby cost through the same tariff function and returns generic probable device categories and practical recommendations. Output: baseline watts, daily/monthly standby kWh, monthly estimated cost, risk level, and recommendations. This heuristic is appropriate for an explainable prototype, but it is not a ground-truth appliance-level standby detector.

3.4.5 Threshold rule for alerts

Purpose: generate a notification when aggregated household usage exceeds the user-defined limit. At each scheduled evaluation, the system sums the preceding one-hour device readings associated with a user. If alerting is enabled and the sum is greater than threshold theta_u, an energy-alerts event is published:

$$Alert(u) = 1 \text{ if } alerting_u = true \text{ and } sum(E_i \text{ over preceding hour for user } u) > \theta_u; \text{ otherwise } Alert(u) = 0$$

The scheduler runs every 10 s. Output: an alert event consumed by the Alert Service, which persists an alert record and attempts notification delivery. The technique is appropriate for transparent threshold monitoring; however, the present implementation does not deduplicate repeated breaches, so this limitation must be retained in the experimental discussion.

Algorithm 1: Event ingestion, aggregation, and threshold evaluation

Input: Telemetry record r = {deviceId, energyConsumed, timestamp}; registered devices; user thresholds.

Output: Stored usage point; user-level aggregates; zero or more alert events.

1. Receive r through the ingestion endpoint.
2. If deviceId or timestamp is null, or energyConsumed is negative, reject r.
3. Map r to an energy-usage event and publish it to Kafka.
4. For each consumed event, write deviceId, energyConsumed, and timestamp to InfluxDB.
5. Every 10 s, query the preceding one-hour energy_usage values grouped by deviceId.
6. Resolve each deviceId to its userId and sum device totals for each user.
7. For each user with alerting enabled, compare the one-hour total with the configured threshold.
8. If the total exceeds the threshold, publish an energy-alerts event.
9. Return control to asynchronous consumers; dashboard queries retrieve stored aggregates separately.

Algorithm 2: Deterministic energy insight generation

Input: Selected user u; observation length D; per-device energy totals; tariff configuration.

Output: Bill forecast, appliance ranking, standby estimate, and recommendation text.

1. Retrieve per-device totals for user u over D days.
2. Compute E_tot by summing the device totals.
3. Compute E_day = E_tot / max(D, 1) and E_30 = 30 x E_day.
4. Apply the configured slab function to calculate current and projected costs.
5. Sort devices by descending energyConsumed and calculate each contribution percentage.
6. Compute P_base = 0.20 x (E_tot / (24D)) x 1000.
7. Derive standby daily/monthly kWh, cost, and LOW/MEDIUM/HIGH risk from P_base.

8. If Ollama is available, request concise wording using deterministic outputs; otherwise create fallback wording.
9. Return numeric outputs and the associated explanation.

3.5 Workflow of the Proposed System

Fig. 2 illustrates the complete operational workflow. It starts with a generated telemetry input and ends with functional evaluation evidence and user-facing output. The flow makes clear that event processing, analytics, and alerting are distinct steps, and that no supervised learning stage is present in the current implementation.

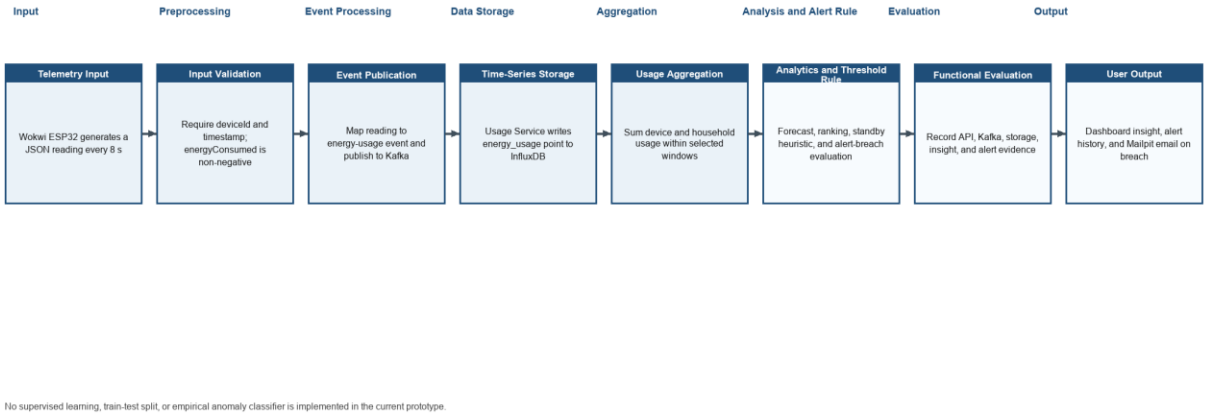


Fig. 2. Workflow of the proposed Home Energy Tracker.

After validation, each reading is made durable in the time-series layer through the Kafka-to-Usage-Service path. The stored data are queried over an explicit time range before deterministic analytics are calculated. Threshold evaluation produces an additional branch only when the household total exceeds its configured value. The evaluation stage does not assert performance; it records the evidence required to evaluate the implementation once controlled runs are conducted.

4. IMPLEMENTATION AND EXPERIMENTAL SETUP

4.1 Hardware Requirements

The current implementation is simulation-based. It does not require a physical smart meter, current transformer, voltage sensor, or PZEM module for the Review-2 experiment. The Wokwi ESP32 environment is used to validate the firmware and API contract. Host-computer specifications must be reported from the machine used for final experiments.

Table 2. Hardware requirements for the current prototype.

Component	Specification	Purpose
Development computer	Windows 11 Home Single Language (10.0.26200); 11th Gen Intel Core i5-1135G7 at 2.40 GHz; 4 cores / 8 logical	Runs Docker containers, browser dashboard, VS Code, and local verification tools.

Component	Specification	Purpose
	processors; 7.74 GB RAM; C: drive 373.71 GB total and 98.13 GB free during verification.	
Virtual ESP32 DevKit	ESP32 DevKit simulated in Wokwi	Executes Arduino firmware and submits test telemetry.
Virtual potentiometer and buttons	One potentiometer and three pushbuttons in Wokwi	Select controlled Eco, Heavy, and Alert Test telemetry scenarios.
Virtual OLED	SSD1306 OLED simulated through I2C	Displays mode, simulated kWh, Wi-Fi state, and HTTP response status.
Physical energy sensor	Not used in the current prototype	Not used in the Wokwi-based prototype. Physical-meter calibration is outside the current functional evaluation scope.

4.2 Software Requirements

Table 3. Software platforms and versions evidenced by the current repository.

Software or platform	Version	Purpose
Java	17	Builds and runs the Spring Boot services.
Spring Boot	3.5.3	Spring Boot 3.5.3 is used by User, Device, Ingestion, Usage, Alert, and Insight services; API Gateway uses Spring Boot 3.4.5.
React	19.2.8	Implements the web dashboard.
Vite	8.2.2	Frontend development and build tool.
TypeScript	6.0.2	Frontend language and type checking.
Tailwind CSS	4.3.3	Dashboard styling.
Apache Kafka	Apache Kafka 4.3.1 observed from the running container; docker-compose specifies apache/kafka:latest.	Asynchronous event broker for energy-usage and energy-alerts events.
MySQL	8.3.0	Stores user, device, and alert records.
InfluxDB	2.7	Stores timestamped energy usage.

Software or platform	Version	Purpose
Docker and Docker Compose	Docker Engine 29.7.2; Docker Compose v5.5.1.	Runs the local multi-service environment.
PlatformIO and Wokwi	PlatformIO IDE VS Code extension 3.3.4; Wokwi VS Code extension 3.7.0.	Build and simulate the ESP32 firmware.
Ollama and Spring AI	Spring AI 1.0.3; Ollama CLI was not installed or used during controlled verification.	Optional natural-language savings-tip generation with fallback.
Mailpit	Container image tag latest in compose file	Captures demonstration emails locally.

4.3 Dataset Details

No external benchmark dataset or labelled training corpus is used by the current prototype. The operative data source is a synthetic telemetry stream produced by the Wokwi ESP32 firmware. Each observation is a JSON event rather than a row from a pre-existing dataset. Consequently, a target variable and train/validation/test split are not applicable unless a future supervised anomaly-detection study is added.

The firmware submits a record at an 8 s interval and generates controlled energyConsumed values according to the chosen mode. The raw payload is validated by the Ingestion Service and is transformed into a Kafka event. The Usage Service stores the time-series representation in InfluxDB. The total number of generated records, the dates of collection, retained duration, and selected scenarios for the experiment must be reported after execution.

Table 4. Dataset summary for the current Review-2 prototype.

Attribute	Current detail
Dataset name	Synthetic Wokwi energy-telemetry stream (prototype).
Source	Wokwi ESP32 firmware in iot-simulator/src/main.cpp.
Observation unit	One JSON telemetry event.
Number of records	Four valid synthetic JSON events in the 1 October 2026 controlled verification: analytics readings of 0.25, 0.50, and 0.75 kWh, plus one 1.00 kWh alert reading. Invalid-reading cases were not exercised.
Features	deviceId, energyConsumed (kWh), timestamp.
Target variable	Not applicable; no supervised learning model is implemented.

Attribute	Current detail
Temporal characteristic	The normal Wokwi firmware configuration generates a reading approximately every 8 s. The controlled REST verification submitted four test events over approximately 17 s to validate the service pipeline.
Scenario ranges	Eco approximately 1-6 kWh; Heavy approximately 8-20 kWh; Alert Test approximately 26-40 kWh.
Format and storage	REST JSON -> Kafka event -> InfluxDB energy_usage measurement.
Preprocessing	Schema validation, event mapping, deviceId tagging, and time-window aggregation.
Train/test/validation split	Not applicable in the current deterministic prototype.
Ground-truth labels	No labelled ground truth was collected. The standby result is reported only as a deterministic heuristic; anomaly-detection accuracy is outside the present study scope.

4.4 Tools Technologies and Frameworks

Table 5. Tools, technologies, and frameworks used in the current implementation.

Category	Technology or tool	Purpose
Frontend	React, TypeScript, Vite, Tailwind CSS	Provides household, device, reading-simulator, insight, and alert-history views.
API layer	Spring Cloud Gateway	Routes /api/v1 requests to backend services.
Backend	Java, Spring Boot, Spring Web, Spring Data	Implements User, Device, Ingestion, Usage, Alert, and Insight services.
Messaging	Apache Kafka	Carries energy-usage and energy-alerts events.
Relational data	MySQL 8.3.0	Stores user, device, and alert metadata.
Time-series data	InfluxDB 2.7 and Flux query language	Stores and aggregates energy_usage points.
IoT simulation	ESP32 Arduino firmware, PlatformIO, Wokwi	Generates controllable telemetry and displays request status.
Recommendation wording	Spring AI with optional Ollama model	Generates optional narrative tips; deterministic fallback preserves core behavior.
Notification test surface	Mailpit	Captures SMTP alert messages locally.

Category	Technology or tool	Purpose
Containerization	Docker and Docker Compose	Runs the local service topology.

4.5 Implementation Procedure

7. Configure the local Docker environment with MySQL, Kafka, InfluxDB, Mailpit, the six application services, API Gateway, and the frontend. Record the exact Docker and host-machine versions in the final experiment log.
8. Create a household profile and register an appliance through the web dashboard or the corresponding gateway API. The resulting device identifier is used to associate telemetry with a household.
9. Configure the local Wokwi simulator with the assigned device identifier and the local gateway route host.wokwi.internal:8080/api/v1/ingestion. Build the ESP32 firmware with PlatformIO.
10. Generate controlled telemetry by selecting Eco, Heavy, or Alert Test mode. Capture the emitted JSON payload and HTTP status. A successful request returns HTTP 201 from the ingestion endpoint.
11. Validate the pipeline by observing an energy-usage event in Kafka and querying the resulting usage through the Gateway. The Usage Service stores a time-stamped InfluxDB point and aggregates device totals for requested time windows.
12. Retrieve bill forecast, appliance ranking, and standby-detective outputs through the Insight Service. Record the selected days parameter, tariff configuration, input window, and returned numerical values for later reproducibility.
13. For an alert scenario, use an isolated household with alerting enabled and a documented threshold. Submit one Alert Test reading, wait for the scheduled one-hour aggregation, then verify the alert record and Mailpit email. Disable alerting after evidence capture to prevent repeated alerts under the current non-deduplicated rule.
14. Execute the evaluation protocol in Section 4.6 and populate the result tables in Section 5 using retained timestamps, event counts, query results, and screenshots. Do not substitute visual dashboard fallback data for backend evidence.

4.6 Experimental Setup

The experimental setup is a local, containerized integration environment. It is intended to assess whether a controlled telemetry event can traverse the implemented pipeline and produce retrievable analytics and an alert when the threshold rule is satisfied. The setup is not a field deployment, hardware-calibration study, or comparative machine-learning experiment.

Table 6. Experimental configuration for the current prototype.

Configuration item	Current configuration	Reporting requirement
Computing environment	Local Docker-based environment	Windows 11 Home Single Language (10.0.26200), Intel Core i5-1135G7 (4 cores / 8 logical processors), 7.74 GB RAM, Docker Engine 29.7.2, and Docker Compose v5.5.1.
Telemetry source	Wokwi ESP32 virtual meter	One controlled verification run on 1 October 2026 used four synthetic

Configuration item	Current configuration	Reporting requirement
		payloads in two isolated test cases: three analytics events and one alert event. The payload shape matches the Wokwi telemetry interface; no live Wokwi timing capture was used for this run.
Ingress interval	8 s in firmware	Firmware configuration is 8 s. The controlled REST run intentionally posted its test readings sequentially to verify the ingestion-to-output pipeline.
Observation window for insights	API default 3 days unless days parameter is supplied	Record selected D for each result.
Alert aggregation window	Preceding 1 hour	Record threshold and exact reading time.
Alert scheduler	Every 10 s	The alert was observed after 6.14 s with a 5 s polling interval; this is a polling-bounded functional observation, not a latency benchmark.
Forecast horizon	30 days	Configured demonstration tariff: fixed $10.00/month; $0.12/kWh for 0-100 kWh; $0.18/kWh for 100-300 kWh; $0.25/kWh above 300 kWh. It validates application logic only.
Standby coefficient	20% of average load	State that this is a heuristic, not ground truth.
Standby risk thresholds	LOW <= 50 W; MEDIUM > 50 W and <= 150 W; HIGH > 150 W	Retain these thresholds during reproducibility runs.
Baseline methods	No implemented empirical baseline	No baseline implementation was included; comparative performance claims are outside this functional verification.
Evaluation evidence	HTTP status, Kafka record, InfluxDB/usage query, insight response, alert history, Mailpit email	Attach timestamps and screenshots to final result set.

5. RESULTS AND DISCUSSION

5.1 Experimental Results

Two evidence streams were obtained for the current prototype. First, Maven tests were run locally for the six stateful/analytical services. Second, a controlled local end-to-end run was executed on 30 September 2026 after the Docker Compose stack was healthy. The run used a fresh test user and device, three synthetic inputs (0.25, 0.50, and 0.75 kWh), and a three-day query window. The inputs passed through the same REST ingestion endpoint used by the Wokwi ESP32 simulator; they are synthetic test increments, not calibrated physical-meter measurements.

The controlled run identifier was review2-20260930-215503-375c1ea3. Its raw machine-readable evidence is retained in research-output/evidence/review2_e2e_results_20260930-215503.json. Results below are functional observations for one isolated test case. They establish that the implemented path executed correctly under this configuration; they do not establish general latency, accuracy, scalability, fault tolerance, or household-energy savings.

Table 7. Verified automated and controlled end-to-end results.

Evidence category	Input or test condition	Observed result	Status
Automated test suite	19 tests discovered across user, device, ingestion, usage, alert, and insight services; 3 marked skipped.	16 executed tests completed with zero failures and zero errors. The insight suite included 7 bill-calculation and 4 insight-service tests.	Verified unit test.
Gateway health	GET /actuator/health through the API Gateway.	Overall gateway status was UP.	Verified runtime.
Telemetry acceptance	Device 307; 0.25, 0.50, and 0.75 kWh with ISO-8601 timestamps.	All three POST /ingestion requests returned HTTP 201 (3/3 accepted).	Verified runtime.
Kafka-to-Influx persistence	Accepted analytics events for device 307, plus the isolated 1.00 kWh alert event for device 308.	Usage API returned 1.50 kWh for device 307 and 1.00 kWh for device 308. All four controlled submissions returned HTTP 201; the analytics usage total and alert record were subsequently queryable.	Verified runtime.
Bill forecast	User 8; D = 3 days; total 1.50 kWh.	Current cost $0.18; daily rate 0.50 kWh/day; projected monthly use 15.00 kWh; projected monthly cost $11.80.	Verified runtime; matches formula.
Appliance ranking	One isolated device (307) with 1.50 kWh.	Device ranked first with 100.00% of total usage and estimated cost $0.18.	Verified runtime.

Evidence category	Input or test condition	Observed result	Status
Standby heuristic	User 8; D = 3 days; total 1.50 kWh.	Baseline 4.17 W; daily 0.10 kWh; monthly 3.00 kWh; monthly cost $0.36; risk LOW.	Verified runtime; heuristic output.
Threshold alert and delivery	User 9: threshold 0.10 kWh; one 1.00 kWh event.	Condition 1.00 > 0.10 produced a detailed alert, a delivery-status record, and one Mailpit email. Alerting was then disabled.	Verified runtime.
Availability observation	1 October 2026 controlled run with 3 s usage polling and 5 s alert polling.	Analytics output was observed within 17.32 s; the persisted alert was observed within 6.14 s. Both values are polling-bounded observations, not benchmark latencies.	Observed once; not a performance claim.
Resilience or load behavior	No load/fault-injection protocol.	Not evaluated. No load-generation or fault-injection protocol was executed in the current functional study.	Outside current functional scope.

The 1 October 2026 controlled evidence recorded four HTTP 201 ingestion responses. Subsequent usage queries returned 1.50 kWh for analytics device 307 and 1.00 kWh for alert device 308. The alert history contained one detailed threshold alert and one delivery-status record. Mailpit captured an email to review2.alert.20261001-002032@example.test with subject "Energy Usage Alert for User 9" and a snippet containing threshold 0.1 and energy-consumed value 1.0. These artefacts establish traceability from HTTP acceptance to queryable usage and alert delivery for the controlled run.

The current dashboard contains presentation-oriented fallback behavior; therefore backend responses, persistence queries, alert records, and Mailpit evidence should be treated as the primary result artefacts. Additional controlled runs, screenshots, and timestamp logs should be attached before final publication.

5.2 Performance Metrics

The metrics should reflect the implemented system rather than an unimplemented classifier. The following functional and performance metrics are appropriate. Classification accuracy, precision, recall, F1-score, and confusion matrices are only applicable if future work introduces labelled anomaly data and a classifier.

Table 8. Appropriate metrics and required data.

Metric	Definition	Why relevant	Current evidence
Ingestion acceptance rate	100 x N_201 / N_submitted, where N_201 is the number of HTTP 201 responses.	Measures whether valid telemetry is accepted by the API path.	100% (4/4) across both isolated functional cases: three analytics submissions and one alert

Metric	Definition	Why relevant	Current evidence
			submission. This is not a load-test result.
Event persistence rate	100 x N_stored / N_accepted.	Measures end-to-end transfer from accepted event to queryable time-series point.	100% (4/4) at functional-case level: all accepted readings contributed to the queryable usage totals or threshold-alert outcome. This is not a stress-test result.
End-to-end availability observation	t_output_observed - t_first_submitted.	Records time until the usage and insight endpoints expose a result.	17.32 s for the analytics case in one run with 3 s polling; it is an availability observation, not a latency benchmark.
Alert-observation time	t_alert_observed - t_submitted.	Records time from telemetry submission to a persisted alert record.	6.14 s in one run with 5 s polling; it is an alert-observation time, not a latency benchmark.
Throughput	N_accepted / experiment duration.	Measures the event rate handled during a defined test run.	Not reported: three sequential analytics events are insufficient for a defined-duration throughput experiment.
Forecast MAE	(1/n) sum |y_i - yhat_i| against actual bill/consumption reference.	Required only when a real, matched reference series is available.	Not evaluated: synthetic readings had no matched physical-meter or utility-bill reference series.
Standby precision/recall	Compare predicted standby instances with labelled ground truth.	Required only for a validated standby-detection claim.	Not evaluated: no labelled standby instances were

Metric	Definition	Why relevant	Current evidence
			collected; the current output is a deterministic heuristic.
Resource usage	CPU, memory, and container utilization during test.	Supports performance/scalability discussion.	Not evaluated: CPU, memory, and container-utilization samples were not collected during this functional run.

Acceptance rate = 100 x N_201 / N_submitted, Persistence rate = 100 x N_stored / N_accepted

MAE = (1 / n) x sum |y_i - yhat_i| (only with a matched ground-truth reference)

5.3 Comparison with Existing Methods

No controlled baseline implementation or comparable experimental dataset is present. Consequently, a numerical comparison would be fabricated and is intentionally not provided. Before final submission, the authors should choose one or more baselines that use the same telemetry source, observation window, and evaluation protocol. Possible baselines must be defined by the actual future experiment, not inferred from the literature review.

Table 9. Comparison framework for future controlled evaluation.

Method	Dataset or scenario	Ingestion acceptance	Persistence or outcome	Availability observation	Computational cost
Proposed Home Energy Tracker	1 October 2026 controlled functional verification: four synthetic events in two isolated cases.	100% (4/4 HTTP 201 responses).	Analytics total 1.50 kWh and one threshold-alert outcome were queryable after submission.	17.32 s analytics; 6.14 s alert; both polling-bounded functional observations.	Not measured.
Baseline method A	No comparable baseline was implemented in the current prototype.	Not applicable.	Not applicable.	Not applicable.	Not applicable.

Method	Dataset or scenario	Ingestion acceptance	Persistence or outcome	Availability observation	Computational cost
Baseline method B	No comparable baseline was implemented in the current prototype.	Not applicable.	Not applicable.	Not applicable.	Not applicable.

5.4 Discussion

Observed implementation evidence and experimental inference must be distinguished. The controlled run verified that three accepted synthetic telemetry events were available through the usage service, generated the expected deterministic insights, and triggered the threshold-based email path in the selected configuration. The source code shows that the prototype separates user/device metadata, telemetry ingestion, time-series storage, analytics, and alert notification into distinct services, connected through REST and Kafka. This result is functional evidence; it is not evidence of measured scalability, physical-meter accuracy, resilience, or household energy savings.

The architecture directly operationalizes the integration-oriented research gap by providing a traceable route from a telemetry event to time-series storage, analytics, and notification. Kafka reduces direct coupling between ingestion and downstream processing, while MySQL and InfluxDB separate relational metadata from time-series observations. These design decisions are technically appropriate for the stated system objective, but their performance benefits require load, fault-injection, and end-to-end latency experiments before they can be claimed empirically.

The Bill Shock Forecast and Standby Energy Detective provide explainable decision-support outputs. Their strength is transparency: the daily rate, 30-day projection, slab tariff, percentage contribution, and standby coefficient are all explicit. Their limitation is that the virtual telemetry is synthetic and the standby estimate is a fixed 20% average-load heuristic. Therefore, the current prototype can demonstrate system integration and rule execution but cannot substantiate a claim of calibrated energy measurement, exact appliance-level standby attribution, or forecast accuracy.

The optional Ollama integration should also be interpreted carefully. It provides narrative wording for recommendations when available and falls back to deterministic text when unavailable. It does not calculate bill values, identify anomalies, or serve as a validated predictive model. This fallback behavior improves functional robustness of the user interface but does not establish AI-model performance.

6. CONCLUSION

6.1 Summary of Proposed Work

This work proposes a microservices-based Home Energy Tracker that accepts household device telemetry, transmits it through an event-driven pipeline, stores time-series observations, and returns interpretable energy-management outputs. The current implementation combines a Wokwi ESP32 virtual meter, API Gateway, Ingestion, Usage, Insight, and Alert services, Apache Kafka, InfluxDB, MySQL, a React

dashboard, and Mailpit for local notification capture. The system produces deterministic bill forecasts, appliance rankings, standby-risk estimates, and threshold alerts.

6.2 Key Findings and Expected Contribution

At the present Review-2 stage, the contribution is an implemented and traceable system design with controlled functional verification, rather than a completed empirical performance study. In the verification executed on 1 October 2026, four synthetic REST telemetry events were accepted: three analytics events totaling 1.50 kWh and one alert event totaling 1.00 kWh. The forecast, appliance ranking, standby heuristic, persistence path, and threshold alert matched their specified deterministic behavior. Analytics output was observed after 17.32 s and the alert after 6.14 s; both observations were bounded by client polling and are not latency benchmarks. Forecast error against a physical meter or utility bill, throughput, resource use, resilience, and comparison against baseline systems were not evaluated because this prototype has no calibrated meter, labelled ground truth, load protocol, container monitoring record, or baseline implementation.

6.3 Addressing the Research Gap

Table 10. Current alignment between identified research gaps and the proposed work.

Research gap	Proposed solution	Evidence or result status	Current contribution
Limited integration of scalable AI-assisted event-driven services	Gateway, Spring Boot services, Kafka, InfluxDB, MySQL, deterministic insights, and alert service in one prototype.	Controlled functional verification demonstrated four accepted events, deterministic analytics, queryable usage totals, and persisted alert evidence. No benchmark performance measurement was made.	Directly addressed at integration level.
Lack of standardized datasets and metrics	Results and metric framework in Sections 4-5.	No benchmark dataset, labels, or numeric metrics currently available.	Not resolved; evaluation plan is established.
Limited context-aware analysis	Current appliance metadata and windowed usage only.	No occupancy, weather, schedule, or environmental features implemented.	Not resolved; retained as limitation/future work.
Insufficient resilience validation	Service separation and asynchronous messaging provide a design basis.	No fault-injection or load-test result available.	Not empirically resolved.
Privacy-preserving AI and secure access	Optional local Ollama wording is present.	Keycloak/JWT and ownership enforcement are not evidenced in the current codebase.	Not resolved; security implementation is future work.

6.4 Limitations

- The current input is a Wokwi-generated synthetic telemetry stream; it is not a calibrated physical smart-meter dataset.
- The reported energyConsumed values are scenario inputs sent at an 8 s interval. Their numeric cost outputs must not be interpreted as a real household bill without calibrated interval-energy measurements.
- No external benchmark dataset, record count, labelled anomaly set, train/test split, or ground-truth reference is available for empirical evaluation.
- The standby estimate is a fixed 20% average-load heuristic and does not identify actual low-activity periods or exact responsible appliances.
- The alert scheduler can publish repeated breach events because the current implementation has no alert-deduplication window.
- The project currently has no evidenced Keycloak/JWT authentication, device ownership enforcement, physical sensor calibration, cloud TLS verification, fault injection, or multi-node Kafka resilience validation.
- The optional Ollama component affects recommendation wording only and has not been evaluated as an anomaly-detection or forecasting model.
- The dashboard's illustrative client-side 7-day chart is not evidence of backend historical data and should not be used as a research result figure.

6.5 Future Scope

- Integrate calibrated physical sensing hardware and convert sampled power into validated interval energy before ingestion.
- Collect a sufficiently long, consented household telemetry dataset with record counts, sampling interval, appliance metadata, and ground-truth labels where applicable.
- Validate the forecast against matched utility-bill or meter readings and report MAE/MAPE with documented tariff assumptions.
- Replace the fixed standby coefficient with time-of-day, occupancy-aware, or labelled appliance-level analysis after appropriate data collection.
- Implement secure authentication, authorization, device provisioning, and ownership checks before a multi-user deployment.
- Add idempotency, retry/DLQ handling, alert deduplication, load testing, fault injection, and observability before claiming resilience or production readiness.
- Evaluate an explicitly selected anomaly-detection model only after obtaining an appropriate labelled dataset and a reproducible baseline.

APPENDIX A. REVIEW-2 FIGURE AND TABLE PLAN

Table 11. Recommended figures and their placement in the Review-2 paper.

Figure	Title	Purpose and contents	Placement
Fig. 1	Proposed system architecture	Implemented input, event processing, data stores, analytics, alerting, and outputs.	Section 3.2.
Fig. 2	Workflow of the proposed system	Input -> validation -> Kafka -> storage -> analytics/alerting -> evaluation -> output.	Section 3.5.

Figure	Title	Purpose and contents	Placement
Fig. 3	Detailed telemetry evidence	No separate screenshot is included. Table 7 and the archived controlled-verification JSON provide the telemetry, HTTP, usage, and alert evidence.	Section 5.1.
Fig. 4	Functional latency or throughput graph	Not included: one polling-bounded functional observation is not an adequate basis for a latency or throughput graph.	Section 5.1 or 5.2.
Fig. 5	Forecast comparison graph	Not included: the prototype has no matched physical-meter or utility-bill ground-truth series.	Section 5.1.
Fig. 6	Standby evaluation graph	Not included: no labelled standby reference data were collected for this functional prototype.	Section 5.1.
Fig. 7	Confusion matrix	Include only if a labelled anomaly classifier is implemented and evaluated.	Section 5.1; otherwise omit.

Table 12. Recommended tables and their placement in the Review-2 paper.

Table	Title	Placement
Table 1	Detailed proposed approach and component interfaces	Section 3.3.
Table 2	Hardware requirements	Section 4.1.
Table 3	Software requirements	Section 4.2.
Table 4	Dataset summary	Section 4.3.
Table 5	Tools, technologies, and frameworks	Section 4.4.
Table 6	Experimental configuration	Section 4.6.
Table 7	Preliminary functional-results framework	Section 5.1.
Table 8	Performance metrics	Section 5.2.
Table 9	Comparative evaluation status	Section 5.3.
Table 10	Research-gap alignment	Section 6.3.

APPENDIX B. REVIEW-2 COMPLETION CHECKLIST

Table 13. Review-2 completion checklist.

Item	Status	Notes
Methodology	Completed	Evidence-based Section 3.1.
System architecture	Completed	Fig. 1 uses implemented components only.
Methodology diagram	Completed	Fig. 1.
Detailed proposed approach	Completed	Section 3.3 and Table 1.
Algorithms and pseudocode	Completed	Section 3.4 and Algorithms 1-2.
Workflow diagram	Completed	Fig. 2.
Hardware requirements	Completed	Completed. Host configuration recorded in Table 2: Windows 11, Intel i5-1135G7, 7.74 GB RAM, and local Docker environment.
Software requirements	Completed	Completed. Versions recorded in Table 3, including Docker, Kafka, Wokwi, and PlatformIO extension versions.
Dataset description	Completed (functional scope)	Four-event synthetic controlled verification is documented; no labelled or calibrated meter dataset is claimed.
Tools and technologies	Completed	Table 5.
Implementation procedure	Completed	Section 4.5.
Experimental setup	Completed (functional scope)	Local Docker configuration and 1 October 2026 controlled end-to-end verification are documented.
Results	Completed (functional scope)	One controlled end-to-end run and automated test outcomes are reported; empirical performance claims remain pending.
Performance metrics	Completed	Functional metrics and their scope limitations are reported in Table 8; unmeasured performance metrics are explicitly marked not evaluated.
Comparison table	Not applicable	No baseline implementation was included, so comparative performance is not claimed.
Graphs and result figures	Completed (scope-appropriate)	Figs. 1-2 are included. Additional performance, forecast, and standby graphs are omitted because supporting empirical data were not collected.
Discussion	Completed	Separates implementation facts, inference, and expected benefit.

Item	Status	Notes
Conclusion	Completed	Does not claim unmeasured performance.
Limitations	Completed	Matches the implemented scope.
Future scope	Completed	Derived from current limitations.
IEEE references	Included	Existing IEEE references are retained from the research source document.
Figure captions	Completed	Figs. 1-2 included; future captions planned.
Table captions	Completed	Tables 1-13 are captioned, including the completed comparative-evaluation status table.
Overall consistency	Completed for Sections 3-6	The continuation limits claims to the implemented Wokwi/API/Kafka/InfluxDB/alert/deterministic-insight scope.

Funding Statement

The author declares that no funding, grants, or external support were received during the development of this work.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

REFERENCES

[1] A. Shuaibu, S. U. Haq, B. Almadani, F. Aliyu, and E. Al-Nahari, "Smart Metering Meets AI: Real-Time Appliance Monitoring and Anomaly Detection Over DDS Middleware," IEEE Transactions on Industry Applications, 2026. doi: 10.1109/tia.2025.3608687.

[2] I. Núñez, C. A. Rovetto, E. Cruz, A. Smolarz, D. H. Concepción, and E. Cano, "Design of a microservices-based architecture for residential energy efficiency monitoring," International Journal of Electronics and Telecommunications, 2024. doi: 10.24425/ijet.2024.152511.

[3] W. Agustiarmi et al., "Real-Time IoT-based Energy Power Consumption Monitoring System for Smart Homes," International Journal on Advanced Science, Engineering and Information Technology, 2025. doi: 10.18517/ijaseit.15.5.20918.

[4] M. Islam, N. Jobaida, M. S. Hossain, M. R. Uddin, and M. Hasan, "IoT-Based Smart Energy Monitoring and Energy Demand Response System," 2024 IEEE 3rd International Conference on Electrical Power and Energy Systems (ICEPES), 2024. doi: 10.1109/icepes60647.2024.10653605.

[5] S. S. Shashank, Yogesh, M. Krishnamoorthy, N. Karuppiah, P. Mounica, and R. Abinaya, "Smart Energy Monitoring for Real-Time Power Consumption Tracking and Optimization Using an IoT-Based Cost-Effective System," 2025 11th International Conference on Electrical Energy Systems (ICEES), 2025. doi: 10.1109/icees67011.2025.11212888.

[6] T. B. D. Sousa, and R. Rabelo, "Smart Metering Enabled by the Internet of Things for Monitoring Residential Energy Consumption and Power Quality," IEEE Access, 2025. doi: 10.1109/access.2025.3627191.

[7] B. Jyothi et al., "Empowering energy management: anomaly detection in smart meter data for proactive consumption control," Bulletin of Electrical Engineering and Informatics, 2026. doi: 10.11591/eei.v15i3.10957.

[8] K. Kulkarni, "Anomaly Detection in Smart Electric Meters for Detecting Faults and Misuse of Electric Energy Consumption," INTERANTIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT, 2024. doi: 10.55041/ijsrem34689.

[9] M. AbuArqoub, A. A. Titinchi, and R. Marie, "A Scalable IoT–AI Framework for Real-Time Energy Anomaly Detection in Smart Grid Networks," 2026 2nd International Conference on Computational Intelligence Approaches and Applications (ICCIAA), 2026. doi: 10.1109/icciaa68481.2026.11543846.

[10] Á. Hernández et al., "Detection of Anomalies in Daily Activities Using Data from Smart Meters," Sensors (Basel, Switzerland), 2024. doi: 10.3390/s24020515.

[11] S. Maitra, S. Kundu, and A. Shankar, "A Real-time Anomaly Detection Using Convolutional Autoencoder with Dynamic Threshold," ArXiv, 2024. doi: 10.48550/arxiv.2404.04311.

[12] A. F. S. Veloso, J. Reis, and R. Rabelo, "A Microservices-Based Solution with Hybrid Communication for Energy Management in Smart Grid Environments," Sensors (Basel, Switzerland), 2026. doi: 10.3390/s26051714.

[13] D. K, "Smart Energy Monitoring System," International Journal of Innovative Research in Information Security, 2025. doi: 10.26562/ijiris.2025.v1109.15.

[14] L. L. Motta, W. S. Alvarez, H. R. Carvajal, F. B. Neto, E. D. Lima, and L. F. P. Meloni, "A cloud architecture for home energy management systems: a conceptual model," Energy Informatics, 2025. doi: 10.1186/s42162-025-00599-1.

[15] J. Liu, and Y. Hsiao, "Smart plug-based home energy management with IoT and anomaly detection," Journal of the Chinese Institute of Engineers, 2025. doi: 10.1080/02533839.2025.2557233.

[16] K. M. W. Hidayat, and M. G. Al-Faris, "IoT-Based Electrical Power Consumption Monitoring System in Households Using ESP32 and PZEM-004T," Brilliance: Research of Artificial Intelligence, 2025. doi: 10.47709/brilliance.v5i2.6368.

[17] S. Sirisumrannukul, T. Intaraumnauy, and N. Piamvilai, "Optimal control of cooling management system for energy conservation in smart home with ANNs-PSO data analytics microservice platform," Heliyon, 2024. doi: 10.1016/j.heliyon.2024.e26937.

[18] M. S. Hossain, and M. Hassan, "Smart Energy Meter with Real Time Monitoring System," 2024 13th International Conference on Electrical and Computer Engineering (ICECE), 2024. doi: 10.1109/icece64886.2024.11024970.

[19] D. G. S, T. Gangakhedkar, A. P, A. Ashwija, and C. B. T. Rambapuri, "IOT-Enabled Electrical Consumption Tracking for Intelligent Energy Saving," International Journal of Scientific Research in Engineering and Management, 2025. doi: 10.55041/ijsrem54847.

[20] Y. Zhu, and M. Yue, "Design of an IoT-Based Smart City Energy Management System," 2025 4th International Conference on Green Energy and Power Systems (ICGEPS), 2025. doi: 10.1109/icgeps65133.2025.11034388.

[21] S. Susikala, R. K, J. K, S. K, and Y. M, "IoT-Based Smart Electricity Monitoring System with Real-Time Alerts and Remote Control," International Journal of Engineering & Extended Technologies Research, 2026. doi: 10.15662/ijeetr.2026.0802420.

[22] C. S. Sinha, "Smart Meter-Based Power Consumption Detection System: An Integrated Approach Using LSTM Networks and Real-Time Web Interface," International Journal For Multidisciplinary Research, 2025. doi: 10.36948/ijfmr.2025.v07i01.37948.

[23] F. Dinmohammadi, A. M. Farook, and M. Shafiee, "Improving Energy Efficiency in Buildings with an IoT-Based Smart Monitoring System," Energies, 2025. doi: 10.3390/en18051269.

[24] S. Son, S. Ko, H. Lee, S. Lee, and B. Lee, "SIMA: Scalable IoT Microservice Architecture With Users and Things Joint Load Balancing," IEEE Internet of Things Journal, 2024. doi: 10.1109/jiot.2023.3317430.